

# Modulo Notion API (`lib/notion.ts`)

Guida tecnica all'integrazione, setup dei database, tipi esportati e pattern d'uso per progetti Next.js multi-ristorante.

## 1. Panoramica del Modulo `lib/notion.ts`

Il file `lib/notion.ts` è un modulo server-side riusabile progettato per astrarre le chiamate REST verso l'SDK ufficiale di Notion (`@notionhq/client`). Si occupa di interrogare i database dei Piatti e dei Menu, gestire la paginazione, normalizzare i dati complessi di Notion in oggetti TypeScript puliti ed eseguire il mapping delle relazioni.

## 2. Configurazione e Prerequisiti (`.env.local`)

Per integrare questo modulo in un nuovo ristorante/progetto Next.js, occorre configurare le seguenti variabili di ambiente:

### Struttura File

```
mio-progetto/  
├── .env.local  
├── lib/  
│   └── notion.ts  
└── app/  
    ├── page.tsx  
    └── menu/page.tsx
```

### File `.env.local`

```
NOTION_TOKEN=secret_xxx  
NOTION_DATABASE_ID=xxx... # Database Menu  
NOTION_PIATTI_DATABASE_ID=xxx... # Database Piatti
```

### ⚠ Regola di Sicurezza & Ambiente Server

Il modulo `notion.ts` deve essere eseguito **esclusivamente nell'ambiente Server**. Non utilizzare il prefisso `NEXT_PUBLIC_` per le chiavi, in modo da prevenire qualsiasi esposizione del token di Notion lato client.

## 3. Schema dei Database Notion Richiesti

Affinché le funzioni di `notion.ts` funzionino correttamente, i database del ristorante su Notion devono rispettare il seguente schema di colonne:

| DATABASE                                                | NOME PROPRIETÀ | TIPO NOTION      | DESCRIZIONE                          |
|---------------------------------------------------------|----------------|------------------|--------------------------------------|
| <b>Piatti Database</b><br>( NOTION_PIAZZI_DATABASE_ID ) | Nome           | Title            | Nome del piatto                      |
|                                                         | Id             | Unique ID / Text | Identificatore unico piatto          |
|                                                         | Categoria      | Select           | es. Antipasto, Primo, Secondo, Dolce |
|                                                         | Descrizione    | Rich text        | Ingredienti / descrizione breve      |
| <b>Menu Database</b><br>( NOTION_DATABASE_ID )          | Nome           | Title            | Nome della categoria/menu            |
|                                                         | Prezzo         | Number           | Costo del menu fisso/ categoria      |
|                                                         | Disponibile    | Checkbox         | Filtro per mostrare solo menu attivi |
|                                                         | Piatti         | Relation         | Relazione verso il DB Piatti         |
|                                                         | Foto           | Files            | Immagine copertina categoria         |

## 4. Interfaccia e Tipi Esportati

Il file esporta le interfacce necessarie per la tipizzazione nei componenti Next.js:

```
export interface PiattoDettaglio {
  idNotion: string;
  Id: string;
  Nome: string;
  Categoria: string;
  Descrizione: string;
}

export interface MenuCategoria {
  idNotion: string;
  Id: string;
  Nome: string;
  Prezzo: number;
  Disponibile: boolean;
  Piatti: PiattoDettaglio[];
  Foto?: string;
}
```

## 5. Funzioni Principali Esportate

```
getTuttiPiatti(): Promise<Record<string, PiattoDettaglio[]>>
```

Recupera tutti i piatti registrati nel database dei piatti e li raggruppa in una mappa con chiave pari alla **Categoria** (es. { "Primo": [...], "Secondo": [...] } ).

```
getMenuCompleto(): Promise<MenuCategoria[]>
```

Recupera tutti i menu/categorie contrassegnati come `Disponibile: true` dal database Menu e risolve le relazioni verso il database Piatti, restituendo la lista completa formattata e tipizzata.

## 6. Pattern di Utilizzo in Altre Pagine / Componenti

### A. In una Pagina Server Component (App Router: `app/menu/page.tsx`)

Esempio di rendering diretto server-side di una lista menu generica:

```
import { getMenuCompleto, MenuCategoria } from "@lib/notion";

export const revalidate = 3600; // Rigenera la pagina ogni ora (ISR)

export default async function MenuListPage() {
  const menuList: MenuCategoria[] = await getMenuCompleto();

  return (
    <section className="container mx-auto py-8">
      <h1 className="text-3xl font-bold mb-6">Menu del Ristorante</h1>
      <div className="grid gap-6">
        {menuList.map((menu) => (
          <div key={menu.idNotion} className="border p-6 rounded-lg shadow-sm">
            <div className="flex justify-between items-center mb-4">
              <h2 className="text-xl font-semibold">{menu.Nome}</h2>
              <span className="text-lg font-bold">€{menu.Prezzo.toFixed(2)}</span>
            </div>
            <ul className="space-y-2">
              {menu.Piatti.map((piatto) => (
                <li key={piatto.idNotion} className="text-sm text-gray-700">
                  <strong>{piatto.Nome}</strong> - {piatto.Descrizione}
                </li>
              ))}
            </ul>
          </div>
        ))}
      </div>
    </section>
  );
}
```

### B. In una Pagina Carta Piatti / Categorie ( `app/carta/page.tsx` )

Esempio di utilizzo di `getTuttiPiatti()` per visualizzare la carta divisa per sezioni:

```
import { getTuttiPiatti, PiattoDettaglio } from "@lib/notion";

export const revalidate = 1800;

export default async function CartaPage() {
  const piattiPerCategoria: Record<string, PiattoDettaglio[]> = await getTuttiPiatti();

  return (
    <main className="max-w-4xl mx-auto p-6">
      <h1 className="text-2xl font-serif text-center mb-8">La Nostra Carta</h1>
      {Object.entries(piattiPerCategoria).map(([categoria, listaPiatti]) => (
        <div key={categoria} className="mb-10">
          <h2 className="text-xl font-bold border-b pb-2 mb-4">{categoria}</h2>
          <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
            {listaPiatti.map((piatto) => (
              <div key={piatto.idNotion} className="bg-white p-4 rounded border">
                <h3 className="font-semibold">{piatto.Nome}</h3>
                <p className="text-xs text-gray-600">{piatto.Descrizione}</p>
              </div>
            ))}
          </div>
        </div>
      ))}
    </main>
  );
}
```

### C. In una API Route interna ( app/api/piatti/route.ts )

Utile nel caso in cui un componente client debba richiedere i dati dinamicamente via client-side fetch:

```
import { NextResponse } from 'next/server';
import { getTuttiPiatti } from '@lib/notion';

export async function GET() {
  try {
    const data = await getTuttiPiatti();
    return NextResponse.json(data);
  } catch (error) {
    return NextResponse.json({ error: "Errore caricamento dati Notion" }, { status: 500 });
  }
}
```

## 7. Checklist di Integrazione per Altri Progetti / Ristoranti

### Passaggi di Handover per un Nuovo Ristorante

1. **Duplicare le Strutture Notion:** Creare o duplicare i due Database su Notion (*Menu* e *Piatti*) assicurandosi che i nomi delle colonne corrispondano a quanto indicato al Punto 3.
2. **Creare l'Integrazione Notion:** Generare un'integrazione interna su [notion.so/my-integrations](https://notion.so/my-integrations) e copiare l'*Internal Integration Token*.
3. **Connettere l'Integrazione:** In Notion, aprire entrambi i Database, cliccare su `...` in alto a destra → `Connections` / `Connessioni` e collegare l'integrazione creata.
4. **Copiare il modulo:** Copiare il file `notion.ts` all'interno di `lib/notion.ts` nel nuovo progetto Next.js.
5. **Popolare .env.local:** Inserire il token e i due ID dei database nel file `.env.local` del nuovo progetto.
6. **Sfruttare le chiamate async:** Invocare `getTuttiPiatti()` o `getMenuCompleto()` direttamente nei Server Components o nelle API Routes di Next.js.